OutSystems ODC Build Rules v2

Reusable rules for building apps on OutSystems ODC with Mentor prompts. Replace <App> with the application's name. v2 adds the lessons from the GUESS CRM test rounds (lines marked **[v2]**).

1. RULES block (paste into every Mentor prompt)
2. UI RULES (paste into every UI-app prompt)
3. SECURITY
4. ARCHITECTURE
5. Working with Mentor
6. Lessons learned [v2] – what went wrong and the rule that prevents it

1. RULES block

Paste inside every Mentor prompt. For a report-only prompt, replace the first line with: **REPORT ONLY. Do not change, add, delete, rename or publish anything.**

```
RULES
- Dependencies are already refreshed in Studio — do NOT refresh.
- Aggregates only, no SQL. Never use CurrDateTime() inside an aggregate filter (put it in a
local variable first).
- Use Settings.X for settings; never hard-code keys, URLs, limits or role codes that belong
in a setting.
- Use <List>.Current inside loops. Compare ids with NullIdentifier(); never compare records
with "=".
- Index() and Substr() are 0-based.
- ListAppend / ListClear are client actions in client flows; never assign one list to another
— copy with a loop.
- Radio Buttons: static options only, each option a direct child of the group, no extra text
inside. Dynamic options use a Dropdown bound to our own options list whose first record is
"Select…", with Empty Value cleared.
- Format date-times on the client (the server runs in UTC).
- Every flow fully connected end to end, no placeholders, no TODO nodes.
- Every exception handler re-raises with Abort Transaction = Yes (or raises the app's own
user exception with the original message).
- The client never sends the acting user id or roles. Server actions build ActingUserId =
GetUserId() and ActingUserRoles from the server roles helper.
- Do not change any CSS (all CSS lives in the Theme app).
- Do not rename or delete public actions or structures; only add inputs or attributes when
asked, and report every caller that needs updating.
- [v2] A returned message never decides success. An action fails only when it raises an
exception or returns an explicit failure output (Success = False, Id = 0).
- [v2] Date Times built on the server are stored in UTC (India time minus 5:30). "Today",
"tomorrow" and weekday names are computed on the server in India time, never left to the AI
model.
- [v2] Seed / bootstrap actions only CREATE missing rows; they never update an existing row.
- [v2] "Latest record" = sort by Id DESCENDING, max 1 record.
- [v2] Never save a row whose mandatory reference is empty or 0 (e.g. a vertical mapping
without a vertical); refuse with a readable message.
- [v2] Database / technical errors never reach the user: log the original message, show a
readable one; validation exceptions pass through unchanged.
- Finish with 0 errors and report every change in a table.
```

2. UI RULES

Paste together with the RULES block into every prompt for the web or mobile UI app.

```
UI RULES
- Build blocks first, then screens. A screen only places blocks and passes parameters; no business logic, forms or tables directly on the screen.
- One block = one job (e.g. LeadForm, ApprovalPanel, AttachmentPanel). Reuse an existing block before creating a new one.
- Blocks get data through input parameters and talk back through events (OnSaved, OnCancel, OnApply). A block never navigates; the screen handles navigation.
- When a block copies input parameters into local variables, it uses a LastSynced guard in OnParametersChanged so a refresh does not overwrite what the user is typing.
- Write calls go through server <Action>AsUser wrappers; screen data comes from server Data Actions. No acting user id or roles sent from the client.
- Every dropdown comes from MasterData / LookupValue (active rows only) through our own options list with a first "Select…" record. No hard-coded options.
- Use only the Theme app's CSS classes (given in the prompt). No inline styles, no new CSS in the app.
- Screens follow the wireframe exactly (layout, labels, order of fields); ask when the wireframe and the flow document disagree.
- Every screen has Roles set; check that each role that needs it can open it (no InvalidPermissions).
- Delete blocks, screens and actions that are no longer used.
- [v2] Set a dropdown's value only AFTER its options list has loaded, and keep the selected record in the list on every rebuild (a value missing from the list falls back to empty).
- [v2] Compare and assign dropdown values with the same type on both sides (Identifier vs Long Integer converted; codes Trim + ToUpper).
- [v2] Prefill precedence: draft value > parent / lead prefill > form default. Apply the draft LAST; defaults and OnChange handlers skip a field the draft already set (one "set by draft" flag per field). Mark a draft applied only after its data has arrived.
- [v2] Feedback messages: Success type for success text, Error only for real errors.
- [v2] After creating a record, navigate to it (or set its id) so a second Save updates it instead of creating another; disable Save while saving.
- [v2] Screens filled by the assistant get an admin-only, collapsed "Draft data" panel listing the received values and the form's final values — it finds most fill bugs in one round.
```

3. SECURITY

```
SECURITY
- Every service action that changes data takes ActingUserId and ActingUserRoles (comma-separated, no spaces, e.g. "ADMIN,RM") and checks owner or role before doing anything; otherwise it raises "Not allowed for your role."
- Role checks match whole codes: Index("," + Roles + ",", "," + Code + ",") <> -1.
- In the UI app, a client action never calls such a service action directly. It calls a server wrapper <Action>AsUser that takes only the business inputs and fills ActingUserId / ActingUserRoles on the server.
- Screen data that needs roles is loaded by a server Data Action that builds the roles itself.
- Lists that take user ids (OwnerIds / UserIds) pass through Scope_FilterUserIds on the server: self + own team; ADMIN and the top commercial role see all.
- Roles fetched on the client are used only to show or hide UI; the server always checks again.
- Workflow callbacks re-read the approval request (latest by Id) and verify its status matches the outcome before acting.
- Demo / seed actions run only if the Foundation flag AllowDemoData = True AND the user is
```

```
ADMIN. Real reference lookups are never behind the demo flag.
- Secrets (API keys, passwords) live only as secret Settings in the ODC Portal, never in code
or chat.
- QA / Prod settings: ShowSampleLogins = False, AllowDemoData = False, all test-only flags
off, test role/hierarchy settings empty.
```

4. ARCHITECTURE

```
ARCHITECTURE
- <App>_Foundation: users, roles helper (GetMyRolesText → RolesCsv, RolesDisplay), hierarchy,
settings flags (Foundation_GetFlags), attachments, notifications, and all lists.
  - LookupValue (Category, Code, Label, SortOrder, IsActive, IsSystem): fixed lists whose
codes drive logic (statuses, types). Admin may change label and order only.
  - MasterData (Type, Code, Name, ParentId, SortOrder, IsActive): editable business masters
(brand, vertical, customer type, terms…). Admin may add, edit and deactivate.
  - No separate static entity per list; records store the Code. One admin catalog lists which
lists are editable.
- <App>_Workflow: generic approval engine (Submit, Act: APPROVE / REJECT / ROLLBACK,
GetMyTasks with DueOn / IsOverdue, DeleteForEntity). Levels, roles, value bands and SLA come
from the ApprovalRule table, editable by the admin.
- Domain apps (e.g. Customer, Sales, Field): own entities + public service actions only; no
UI.
- <App>_Integration: external APIs (keys from secret Settings, daily call-limit guard).
- <App>_Agent (optional): agentic app with one entry point and screen context (entity name +
id); agents fill drafts or make proposals, the user always saves or confirms. Names are
resolved to ids/codes on the server. Tool inputs all mandatory, one DetailsJson where many
fields are needed. Public payload structures are never deleted, recreated or renamed.
- <App>_Theme: all CSS. Prompts never touch CSS. [v2] Web and mobile never share a class name
(web gl-*, mobile gl-m-*).
- <App> (web UI) and <App> Field (mobile): screens and blocks only, calling domain apps
through AsUser wrappers.
- [v2] A service action in another app commits on its own. A callback after it must be safe
to run twice (idempotent), and an ADMIN-only "apply result again" action exists for records
left half-done.
- Publish order: Foundation → Workflow → domain apps → Integration → Agent → UI apps.
```

5. Working with Mentor

- **One session = one small task.** Each prompt names a single app: "Module: <App> → <area>".
- **Open a new Mentor session after every publish.** An old session can revert your changes.
- Mentor never publishes. Publish and refresh dependencies yourself in Studio, and write "already refreshed – do NOT refresh" in the prompt.
- Verify every Mentor report in Studio. It sometimes reports work as done that was not done.
- Before building a new feature, run a **report-only** prompt to learn what the code does today.
- Split large work into batches of 3–7 calls. End each batch with "final scan, list what remains".
- Mentor cannot see the Theme app's CSS, so give the class names in the prompt, and check that every class Mentor used exists in the Theme.
- In every prompt for the agent app, include the PUBLIC CONTRACT rule (payload structures stay Public = Yes; attributes may only be added) and ask for a "Structure | Public" table as the last step.
- **[v2]** When Mentor shows a plan with Proceed / Discard, read it before pressing Proceed – correct it in the same session if it overwrites data, removes a needed branch or weakens a rule.

- **[v2]** When a fix "should work" but the screen still fails, stop guessing: ask Mentor to print the values on screen (admin-only line), then fix the link that breaks.
- **[v2]** Test assistant fixes with a NEW draft – an old draft keeps the values it was stored with.
- **[v2]** After a publish that adds a mandatory input, refresh and fix every consumer app before testing (error "A valid expression must be set for parameter …").

6. Lessons learned [v2]

#	What went wrong	Cause	Rule now
1	"Lead created successfully." shown in red; confirmed proposals reported "1 failed"	Any returned message text was treated as an error	Success is decided by exception / failure output, never by message text
2	Assistant drafts opened with customer, vertical, location or objective empty	Values set before the dropdown lists loaded; form defaults ran after the draft	Set values after lists load; draft applied last; defaults skip draft-set fields
3	Draft had VerticalId 73 but dropdown stayed "Select…"	Identifier compared with Long Integer	Same type on both sides; codes Trim + ToUpper
4	Saved customer disappeared from the lead form	Id set before its option was in the list, so the dropdown reset it to 0	Load the record, rebuild the list with it, then set the value
5	Follow-up at 10:00 would show as 3:30 PM	India time stored as if it were UTC	Server Date Times stored in UTC
6	"Friday" became Thursday	AI model counted the date	Server computes relative dates in India time
7	Customer approved in workflow but still "Pending approval"	Callback failed after the workflow app had already committed; check read the oldest request	Idempotent callbacks, latest request (Id desc), admin recovery action
8	Admin rule changes would be undone by the next bootstrap	Seed updated existing rows	Seeds create only
9	Duplicate-key database error shown to the user	Save always created child rows instead of upserting	Upsert by natural key; readable messages; raw error only in the log
10	"Vertical 0" in dropdowns	A new-customer branch saved a mapping with no vertical	Never save a row with an empty mandatory reference
11	Approver saw no tasks	Role match only when no user was assigned (empty id ≠ null)	Visibility = assigned OR role OR ADMIN, same as the Act authorization
12	Web forms showed a full-height grey column	Mobile page class gl-field reused on web fields	Unique class names per channel
13	Header buttons overlapped when the assistant panel opened	CSS forced buttons onto one line	Header actions wrap; icon buttons keep icon + text on one line